

UNIVERSIDAD TECNOLÓGICA NACIONAL

Tecnicatura Universitaria en Programación a Distancia

Facultad Regional Mendoza

Trabajo Práctico Integrador Food Store

Sistema de Gestión de FOOD STORE en Consola con Persistencia JDBC

Asignatura:	Programación II
Estudiante:	Tiziano Valentini y Juan Martin Garcia
Institución:	UTN - Facultad Regional Mendoza
Fecha de Entrega:	Junio 2026

Índice General

1. Introducción y Alcance del Proyecto	3
2. Marco Teórico y Tecnologías Aplicadas	4
3. Decisiones Técnicas y Arquitectura del Sistema	5
4. Estructura de Paquetes y Código Fuente	6-7
5. Reglas de Negocio, Validaciones y Manejo de Transacciones	8-9
6. Dificultades Técnicas y Soluciones Implementadas	10
7. Conclusiones y Pruebas de Ejecución	11
8. Bibliografía y Webgrafía	12

1. Introducción y Alcance del Proyecto

El presente informe técnico documenta el diseño, la arquitectura y la implementación del sistema **Food Store**, desarrollado como Trabajo Práctico Integrador para la asignatura Programación II de la Tecnicatura Universitaria en Programación (UTN-FRM). El objetivo principal de este proyecto es consolidar los conceptos avanzados de la Programación Orientada a Objetos (POO), el manejo robusto de excepciones estructuradas, la división arquitectónica por capas independientes y la persistencia de datos relacionales mediante una interfaz controlada por consola.

El alcance del sistema abarca la gestión integral de un negocio gastronómico automatizado. Permite la administración mediante operaciones de alta, baja, modificación y consulta (CRUD) sobre cuatro dominios esenciales interconectados: Categorías, Productos, Usuarios y Pedidos. Toda la navegación e interacción con el operador se ejecuta a través de un sistema jerárquico de menús interactivos por consola, validando activamente cada entrada para prevenir estados inconsistentes del software.

2. Marco Teórico y Tecnologías Aplicadas

Para cumplir con las exigencias académicas y asegurar la extensibilidad del código, se aplicaron un conjunto de tecnologías y paradigmas de ingeniería de software que garantizan la robustez de la solución:

- **Java Development Kit (JDK 21):** Se seleccionó la versión de soporte de largo plazo más moderna para aprovechar estructuras idiomáticas avanzadas, tales como la expresión `switch` mejorada con asignación directa (pattern matching básico) y la API de Streams para el procesamiento funcional de colecciones.
- **Java Database Connectivity (JDBC):** En lugar de restringir el ciclo de vida de los datos puramente a la memoria volátil del runtime, se optó por una arquitectura de almacenamiento persistente real basada en JDBC, comunicando la aplicación de forma directa con un motor relacional a través del driver nativo `com.mysql.cj.jdbc.Driver`.
- **Paradigma de Programación Orientada a Objetos (POO):**
 - *Encapsulamiento:* Clases con modificadores de acceso restrictivos, protegiendo el estado interno mediante getters/setters deliberados y constructores parametrizados específicos.
 - *Herencia y Abstracción:* Implementación de una clase madre abstracta denominada `Base`, que centraliza los atributos transversales de auditoría e identidad (ID, marcas de tiempo y estados de borrado lógico) que heredan todas las entidades del dominio.
 - *Polimorfismo:* Sobrescritura sistemática de los métodos estructurales de la clase `Object` (`toString`, `equals`, `hashCode`) e implementación formal de firmas de comportamiento mediante interfaces de contrato lógico (interfaz `Calculable`).
- **Manejo de Excepciones Checked:** Reemplazo del flujo genérico de errores por tipos de excepciones explícitas y controladas, forzando arquitectónicamente a las capas superiores a capturar, tipificar y mostrar mensajes descriptivos al operador sin interrumpir la ejecución del sistema.

3. Decisiones Técnicas y Arquitectura del Sistema

El diseño del sistema responde a una arquitectura desacoplada por capas o niveles de abstracción. Esta decisión técnica garantiza la separación de responsabilidades, facilitando el mantenimiento y permitiendo que modificaciones en la capa de datos o de presentación no afecten la estabilidad de las reglas de negocio esenciales.

Principio de Alta Cohesión y Bajo Acoplamiento

Cada componente posee una única razón para cambiar. Los servicios de negocio desconocen por completo si la interfaz que los invoca es de consola, web o gráfica; de igual manera, las entidades de dominio modelan el negocio de manera pura sin contaminarse de lógica relacional SQL o de lectura de teclado.

Descripción de las Capas Arquitectónicas

La aplicación se organiza jerárquicamente en cuatro niveles independientes:

1. **Capa de Presentación / UI (Clase Main):** Actúa como el único punto de entrada de datos a través de la consola empleando la clase `Scanner`. Se encarga de formatear la salida del sistema, capturar errores de tipo de dato de entrada (vía `NumberFormatException`) y presentar los menús iterativos en base a las opciones seleccionadas.
2. **Capa de Servicios de Lógica (Service):** Es el núcleo operativo o cerebro del sistema. Valida las restricciones lógicas y de negocio (por ejemplo, unicidad de correos, stocks positivos). Si las validaciones se superan, delega la persistencia a la capa inferior. Comunica los errores hacia la UI lanzando excepciones propias de negocio.
3. **Capa de Acceso a Datos (DAO):** Interfaces y clases concretas de implementación JDBC (`Data Access Object`) encargadas exclusivamente de la traducción orientada a objetos hacia sentencias puras en lenguaje SQL relacional. Gestiona la apertura y el cierre seguro de recursos físicos.
4. **Capa de Dominio / Entidades (Entities):** Clases puras de objetos que representan los elementos del modelo real (`Usuario`, `Producto`, `Pedido`, `DetallePedido`, `Categoria`) estructurados bajo el patrón de herencia de la clase abstracta común.

4. Estructura de Paquetes y Código Fuente

De acuerdo con la recomendación de la cátedra, el árbol de código fuente se modularizó rigurosamente en paquetes semánticos definidos. A continuación, se presenta la tabla descriptiva que asocia cada paquete a las clases provistas en la entrega del sistema:

Paquete	Clases Implementadas	Responsabilidad Técnica en el Sistema
config	ConexionDB	Administración central de los parámetros de conexión JDBC de MySQL y provisión segura de la conexión física activa.
dao	CategoriaDAO, CategoriaDAOImpl, ProductoDAO, ProductoDAOImpl, UsuarioDAO, UsuarioDAOImpl, PedidoDAO, PedidoDAOImpl	Encapsulamiento del mapeo relacional (ORM manual), ejecuciones de PreparedStatement, transacciones y lectura de ResultSet.
entities	Base, Calculable, Categoria, Producto, Usuario, Pedido, DetallePedido	Modelado del grafo de objetos de dominio, lógica de constructores por herencia y contratos matemáticos formales.
enums	Rol, Estado, FormaPago	Definición formal de los dominios restrictivos admitidos por el negocio para roles, estados de envío y modalidades transaccionales.
exception	EntidadNoEncontradaException, ReglaNegocioException	Tipificación personalizada del árbol de excepciones checked del sistema para desacoplar el flujo de errores del runtime.
service	CategoriaService, ProductoService, UsuarioService, PedidoService	Validación de restricciones previas, control del stock disponible y orquestación de flujos lógicos de negocio.

Ejemplo Estructural: La Entidad Base Abstracta

Para cumplir de manera uniforme con el requerimiento de baja lógica y auditoría temporal, se implementó la abstracción de identidad en la raíz de las entidades:

```
package entities;
import java.time.LocalDateTime;

public abstract class Base {
    protected Long id;
    protected boolean eliminado;
    protected LocalDateTime createdAt;

    protected Base() {
        this.eliminado = false;
        this.createdAt = LocalDateTime.now();
    }
    // Getters y Setters parametrizados...
}
```

5. Reglas de Negocio, Validaciones y Manejo de Transacciones

El sistema valida estrictamente las reglas mínimas operativas que garantizan la consistencia total del dominio culinario frente a datos maliciosos o erróneos.

Validaciones de Negocio Implementadas

- **Control Numérico Estricto:** En la capa `ProductoService`, previo a persistir un alta o edición, se evalúa mediante comparaciones de signo que el precio y el inventario sean cuantitativamente válidos. Si el precio es menor a cero o el stock es negativo, el sistema interrumpe el flujo arrojando una `ReglaNegocioException`.
- **Garantía de Unicidad de Identidad Digital:** Durante el registro o modificación de un usuario, el método `validarMailUnico` recorre de forma optimizada la persistencia mediante consultas indexadas en la base de datos. Si se detecta una coincidencia con un correo existente perteneciente a otro ID, se detiene la ejecución informando el conflicto al operador de manera legible.
- **Restricciones de Integridad del Pedido:** Es de carácter obligatorio que un pedido se asocie a un usuario real preexistente y no eliminado. Adicionalmente, las cantidades unitarias cargadas dentro de los detalles del pedido deben ser taxativamente superiores a cero unidades.

Manejo Avanzado de Transacciones Relacionales (ACID)

Uno de los puntos más críticos implementados es la persistencia de la estructura Cabecera-Detalle de los pedidos gastronómicos. Dado que un pedido está compuesto por múltiples líneas de `DetallePedido`, se programó un manejo manual y estricto de transacciones a nivel de conexión JDBC en la clase `PedidoDAOImpl` para respetar el principio de atomicidad:


```
con = ConexionDB.getConnection();
con.setAutoCommit(false); // Apertura de transacción manual

// 1) Persistencia de la cabecera del Pedido
psPedido.executeUpdate();

// 2) Iteración y persistencia de cada DetallePedido asociado
for (DetallePedido detalle : pedido.getDetalles()) {
    psDetalle.executeUpdate();
}
con.commit(); // Consolidación atómica de todos los cambios
```

Si ocurre cualquier error técnico o violación de claves foráneas durante la inserción en cascada de los detalles, el bloque `catch` captura la excepción física y activa de manera inmediata un proceso de `con.rollback()`, revirtiendo por completo cualquier registro parcial en la base de datos para no corromper el historial del negocio.

6. Dificultades Técnicas y Soluciones Implementadas

A lo largo del ciclo de vida del desarrollo surgieron diversos obstáculos lógicos derivados del desacoplamiento de capas y el mapeo manual de las relaciones estructurales orientadas a objetos a tablas de un motor relacional SQL.

Dificultad 1: Mapeo manual de grafos de objetos bidireccionales y joins complejos

Problema: Al recuperar un `Producto` desde la base de datos, la tabla de forma nativa solo almacena el campo ID numérico de la categoría (`categoria_id`). Sin embargo, el modelo orientado a objetos exige que la propiedad sea una instancia completa de la clase `Categoria` para poder renderizar correctamente la información a través del método `toString()`.

Solución: Se diseñaron consultas SQL estructuradas basadas en operaciones de acoplamiento interno (`INNER JOIN`), permitiendo extraer de forma unificada los campos de ambas tablas en un único `ResultSet`. Luego, mediante el método privado de infraestructura `mapear(ResultSet rs)`, se instancia primero la categoría de manera aislada y se inyecta por composición directamente en el constructor completo del objeto `Producto`.

Dificultad 2: El Desafío del Soft Delete (Baja Lógica) en Entidades Relacionadas

Problema: La regla impuesta de no destruir físicamente registros (Soft Delete) genera un problema lógico de integridad referencial. Si un operador elimina lógicamente una categoría, los productos asociados en la base de datos continúan apuntando a ella, lo cual podría provocar errores en cascada o renderizados inválidos al listar el inventario general si la categoría ya no se considera activa.

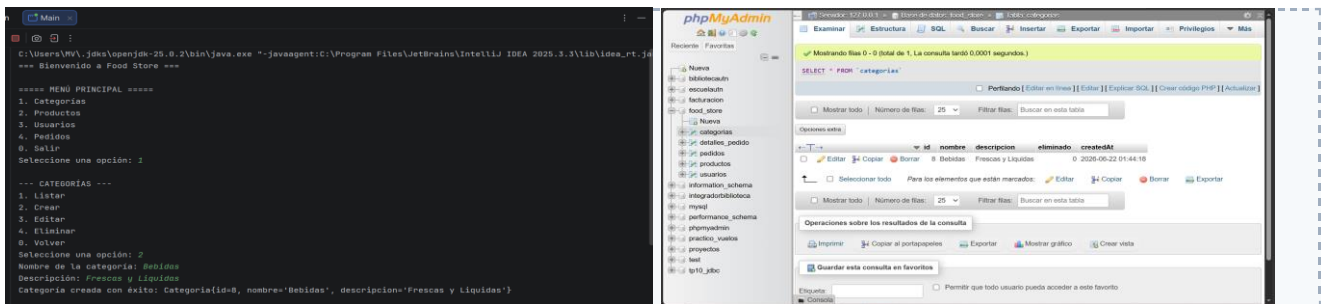
Solución: Se establecieron políticas defensivas en la capa de control. En las consultas generales de listado, se añadió de manera uniforme la cláusula condicional restrictiva `WHERE eliminado = false`. Además, antes de procesar transacciones críticas, el sistema invoca de forma obligatoria al método de búsqueda previa, el cual arroja una excepción controlada si el registro posee su bandera de borrado lógico activa, impidiendo que elementos inactivos sean asignados a nuevos flujos comerciales.

7. Conclusiones y Pruebas de Ejecución

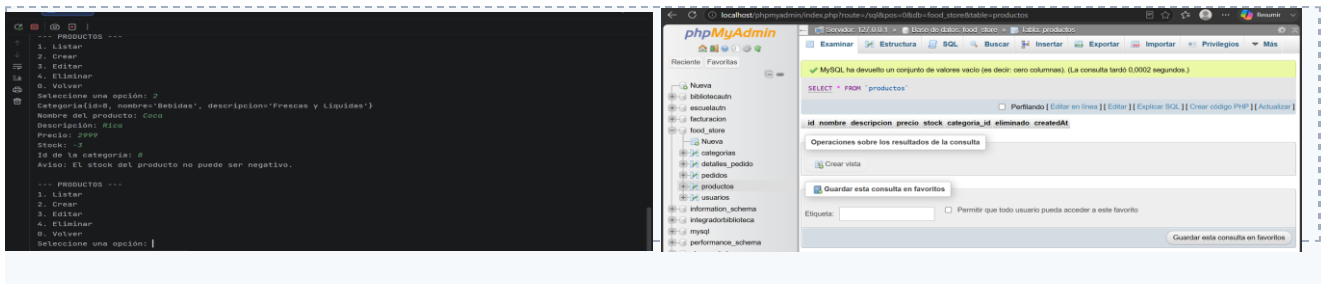
La culminación del sistema **Food Store** demuestra de manera sólida la viabilidad de diseñar arquitecturas de software modulares y altamente legibles bajo el lenguaje Java sin requerir frameworks invasivos externos. Se logró implementar con éxito un flujo transaccional robusto que salvaguarda la información crítica del negocio culinario frente a fallas imprevistas de infraestructura mediante reversiones atómicas.

Las validaciones incorporadas a nivel de lectura por consola aseguran que el operador mantenga una experiencia libre de caídas del runtime provocadas por ingresos accidentales de caracteres alfanuméricos en campos monetarios o de ID. La correspondencia exacta de paquetes y el cumplimiento estricto del backlog mínimo proveen una base de código limpia, mantenible y alineada.

DATOS BIEN CREADOS



DATOS INVALIDOS = NO CREACION



8. Bibliografía y Webgrafía

Para el sustento conceptual y de desarrollo del proyecto se consultaron las siguientes fuentes oficiales de referencia técnica:

1. Universidad Tecnológica Nacional. (2026). *Material de Cátedra y Apuntes Teóricos de Programación II*. UTN FRM.
2. **Enlace al repositorio GitHub:** <https://github.com/juanmagarcia/TPI-PRO2-4-GARCIA-VALENTINI/tree/master>